

Efficient Deployment of Large Language Model across Cloud-Device Systems

Fan Yang^{1,2*}, Zehao Wang^{1*}, Haoyu Zhang¹, Zhenhua Zhu¹, Xinhao Yang¹, Guohao Dai³, Yu Wang^{1†}

¹Tsinghua University, ²SenseTime Inc., ³Shanghai Jiao Tong University

[†]Corresponding author: yu-wang@tsinghua.edu.cn

Abstract—The capabilities of large language models (LLMs) in text comprehension and generation are advancing artificial intelligence. However, the growing number of parameters and computational demands challenge the efficient deployment of inference services. High-performance GPU clusters in the cloud can meet these requirements but incur high service costs and network stability issues, which struggle to meet service-level agreements (SLAs). The “cloud-device collaboration” approach leverages the heterogeneous hardware on both the cloud and device sides to satisfy SLAs efficiently. However, the varying operational intensity among different LLM operators and their dynamic nature complicate load scheduling for cloud-device systems.

To address these challenges, we optimize LLM inference deployment on cloud-device systems through three aspects: scheduling algorithm, hardware modeling, and compilation deployment. For the scheduling algorithm, we analyze the LLM computation network, evaluate the computation-to-memory access ratio under different sequence lengths, and propose a greedy algorithm-based operator-level scheduling strategy. For the hardware modeling, we establish a relationship between operational intensity and GPU resource utilization to estimate operator running time. Finally, we designed a cloud-device LLM compiler framework for quantitative evaluation and efficient deployment across various hardware combinations and inference tasks. In specific inference scenarios, our framework satisfies the need for inference latency and achieves an average cost reduction of 20.7% compared to cloud-side-only inference.

Index Terms—Large Language Models, Cloud-Device Collaboration System, Efficient Inference

I. INTRODUCTION

In recent years, large language models (LLMs) have revolutionized natural language processing tasks, such as language understanding, generation, and question answering, achieving widespread adoption across various industries [1]–[3]. However, the substantial computational requirements and parameter sizes of LLMs pose significant challenges for their efficient deployment. For example, GPT-3 released by OpenAI [4] has 175 billion parameters and requires about 350GB of memory (using FP16 data type), with approximately 660TOPs needed to complete one inference. Despite the efforts to optimize LLMs through techniques such as quantization and pruning to reduce the computational and memory requirements for inference, the end-side devices are still hard to deploy these LLMs. High-performance GPU clusters in the cloud can satisfy these requirements but incur significant hardware costs and rely on stable network connectivity. This dependency can hinder real-time performance and limit service accessibility in

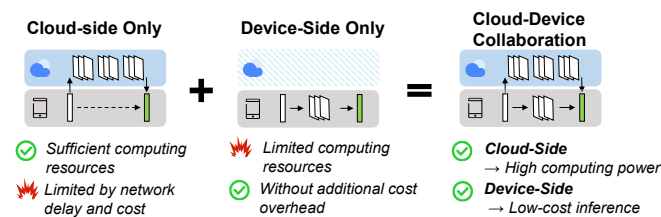


Fig. 1. device-side-only, cloud-side-only and cloud-device collaborative inference schemes.

remote or disconnected environments, which means a struggle to meet service-level agreements (SLAs).

To address the limitations of cloud-side and device-side, a cloud-device heterogeneous system can be adopted to efficiently deploy LLMs, as shown in Fig. 1. For Convolutional Neural Networks (CNNs), some efforts have been made to optimize the deployment on cloud-device systems, typically through large-small model collaboration and computation offloading [5]. Large-small model collaboration relies on techniques such as early stopping, where only a portion of the model is computed under constrained resources. However, this approach is not suitable for current LLMs. Computation offloading, which includes layer-wise and tensor-wise partitioning strategies, also fails to achieve optimal performance for LLMs. The diverse characteristics (e.g., computation-to-memory access ratios) of operators and the dynamism of workload characteristics (e.g., the length of input and output texts) make effective partitioning and offloading difficult. It is necessary to propose an approach to adaptively partition and distribute computational tasks based on real-time workload demands and resource availability.

In this paper, we propose a framework to efficiently deploy LLMs inference services in cloud-device systems under diverse hardware configurations and varying inference workloads. This framework divides computational tasks at the operator level for scheduling. Specifically, we conduct a quantitative analysis of the computation-to-memory access ratio of different operators (e.g., attention and linear layer) in LLMs and their dynamic changes relative to the input and output text lengths. Based on this analysis, we propose a cost- and latency-aware greedy-based scheduling algorithm. To evaluate the execution time of operators efficiently, we develop a hardware simulation model that considers bandwidth, computing power, and the computation-to-memory access ratio. Finally, we design a compilation framework that enables rapid evaluation and efficient deployment across different hardware combinations and dynamic inference tasks.

大规模语言模型在云 - 设备系统中的高效部署

范洋^{1,2*}, 王泽昊^{1*}, 张昊宇¹, 朱振华¹, 杨新浩¹, 戴国豪³, 王烨^{1†} 清华大学,² 商汤科技有限公司,³ 上海交通大学

[†]通讯作者: yu-wang@tsinghua.edu.cn

摘要——大型语言模型（LLM）在文本理解和生成方面的能力正在推动人工智能的发展。然而，参数数量的不断增加和计算需求的增长给推理服务的高效部署带来了挑战。云端高性能 GPU 集群可以满足这些需求，但会产生高昂的服务成本和网络稳定性问题，从而难以满足服务水平协议（SLA）。“云 - 设备协作”方法利用云端和设备端的异构硬件来高效满足 SLA。然而，不同 LLM 操作符之间的操作强度差异及其动态性使得云 - 设备系统的负载调度变得复杂。

为应对这些挑战，我们通过三个方面优化云设备系统上的大语言模型（LLM）推理部署：调度算法、硬件建模和编译部署。在调度算法方面，我们分析了 LLM 计算网络，评估了不同序列长度下的计算与内存访问比，并提出了一种基于贪心算法的算子级调度策略。在硬件建模方面，我们建立了操作强度与 GPU 资源利用率之间的关系，以估算算子运行时间。最后，我们设计了一个云设备 LLM 编译器框架，用于在不同硬件组合和推理任务中进行定量评估和高效部署。在特定推理场景中，我们的框架满足推理延迟需求，并相比仅云端推理平均实现了 20.7% 的成本降低。

索引词 — 大型语言模型、云 - 设备协作系统、高效推理

一、引言

近年来，大型语言模型（LLM）已经彻底改变了自然语言处理任务，例如语言理解、生成和问答，并在各个行业得到了广泛的应用 [1]–[3]。然而，LLM 的巨大的计算需求和参数规模为其高效部署带来了重大挑战。例如，OpenAI 发布的 GPT-3[4] 拥有 1750 亿个参数，使用 FP16 数据类型时大约需要 350GB 的内存，并且完成一次推理大约需要 660TOPs。尽管人们通过量化和剪枝等技术努力优化 LLM，以减少推理的计算和内存需求，但终端设备仍然难以部署这些 LLM。云端的高性能 GPU 集群可以满足这些需求，但会产生高昂的硬件成本，并依赖稳定的网络连接。这种依赖可能会阻碍实时性能，并

*两位作者对这项工作贡献相同。

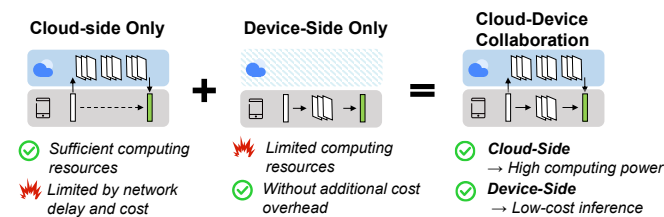


图 1. 仅设备端、仅云端以及云 - 设备协同推理方案。

偏远或断开的环境，这意味着难以满足服务水平协议（SLA）。

为了解决云端和设备端的限制，可以采用云 - 设备异构系统来高效部署大语言模型（LLM），如图 1 所示。对于卷积神经网络（CNN），已经有一些努力用于优化其在云 - 设备系统上的部署，通常通过大 - 小模型协作和计算卸载实现。大 - 小模型协作依赖于诸如提前停止（early stopping）等技术，在资源受限的情况下仅计算模型的一部分。然而，这种方法并不适用于当前的 LLM。计算卸载，包括按层和按张量的分区策略，也未能实现 LLM 的最佳性能。操作符的多样化特性（例如计算与内存访问的比率）以及工作负载特性的动态性（例如输入和输出文本的长度）使得有效的分区和卸载变得困难。因此，有必要提出一种自适应方法来

在本文中，我们提出了一个框架，用于在多样的硬件配置和变化的推理工作负载下，在云 - 设备系统中高效部署大语言模型（LLM）的推理服务。该框架在算子层面划分计算任务以进行调度。具体来说，我们对大语言模型中不同算子（如注意力层和线性层）的计算与内存访问比率及其相对于输入和输出文本长度的动态变化进行了定量分析。基于这一分析，我们提出了一种兼顾成本和延迟的贪心调度算法。为了高效评估算子的执行时间，我们开发了一个硬件模拟模型，该模型考虑了带宽、计算能力以及计算与内存访问比率。最后，我们设计了一个编译框架，使得能够在不同硬件组合和动态推理任务下快速评估和高效部署。

*Both authors contributed equally to this work.

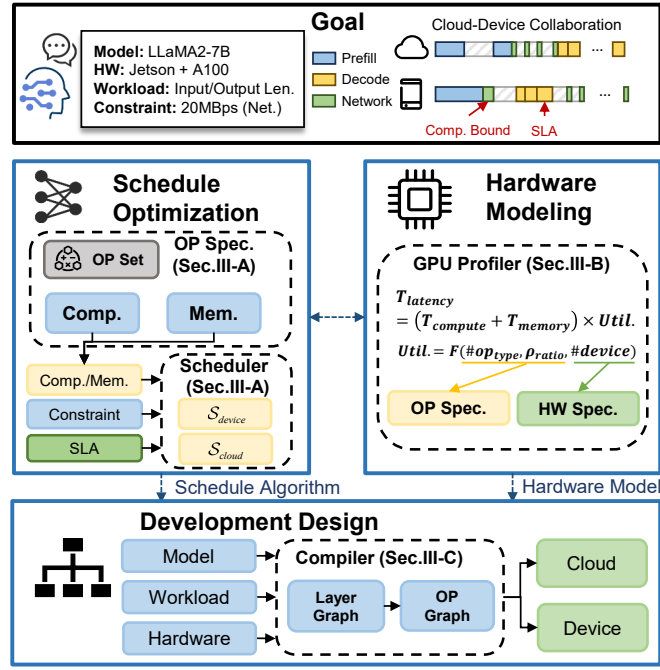


Fig. 2. The overview of our design. Our goal is to ensure that the inference latency (TTFT and TPOT) is within the inference SLA, while minimizing the inference cost.

We demonstrate an overview of this framework in Fig. 2 and summarize the contributions as follows:

- 1) We introduce a novel framework for efficient deployment of LLMs in cloud-device systems. Specifically, we partition the computation load at the operator granularity and propose a cost-latency balancing scheduling algorithm based on greedy algorithm.
- 2) We develop a hardware simulation model that enables rapid evaluation of the computation time for operators with varying computation-to-memory access ratios across diverse hardware platforms.
- 3) We present a compilation framework that facilitates quantitative evaluation and efficient deployment of diverse inference tasks and cloud-device hardware combinations. Experimental results demonstrate that our approach satisfies the need for inference latency and reduces costs up to 20.7% compared to cloud-side-only deployment.

II. BACKGROUND AND RELATED WORK

A. Large Language Models (LLMs) Inference

In this section, we introduce the LLMs inference computation workflow and SLAs. A standard LLM generative inference process comprises two stages [6]: i) the *prefill stage*, where a given prompt sequence is used to generate the key/value cache (KV cache) for each transformer layer within the LLM; and ii) the *decode stage*, which leverages and updates this KV cache to produce tokens one by one, with the current token generation depending on previously generated tokens. In both stages, the generation of each token requires passing through the entire model, typically constructed by stacking multiple transformer blocks [7]. Transformer blocks

comprise two primary operators: the attention operator and the Feed-Forward Network (FFN) operator.

During *prefill stage*, denote the input features of the i -th layer as $X_i \in R^{b \times s \times h}$, where b denotes the batch size, s denotes the input sequence length, and h denotes the hidden dimension of the transformer. For attention operator, the computation is:

$$Q_i = X_i W^{Q_i}, K_i = X_i W^{K_i}, V_i = X_i W^{V_i} \quad (1)$$

$$A_i = \text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{h}}\right) V_i, O_i = A_i W^{O_i} + X_i \quad (2)$$

where $W^{Q_i}, W^{K_i}, W^{V_i}, W^{O_i} \in R^{h \times h}$ are weight matrices. The computation equation of FFN operator is:

$$X_{i+1} = f_{act}(O_i W_1^i) W_2^i + O_i \quad (3)$$

where $W_1^i \in R^{h \times h'}$ and $W_2^i \in R^{h' \times h}$ are weight matrices (h' is the hidden dimension of the second MLP layer).

During *decode stage*, each iteration is designed only to consider the current token. Different from *prefill stage*, the matrix-vector multiplications (MV) operations are the majority for decoding instead of the matrix-matrix multiplications (MM). Denote the feature of the current generated token in the i -th layer as $t_i \in R^{b \times 1 \times h}$. The equations for this process are below:

$$q_i = t_i \cdot W^{Q_i}, k_i = t_i \cdot W^{K_i}, v_i = t_i \cdot W^{V_i} \quad (4)$$

$$K_i \leftarrow \text{Concat}(K_i, k_i), V_i \leftarrow \text{Concat}(V_i, v_i) \quad (5)$$

$$a_i = \text{softmax}\left(\frac{q_i K_i^T}{\sqrt{h}}\right) V_i, o_i = a_i \cdot W^{O_i} + t_i \quad (6)$$

$$t_{i+1} = f_{act}(o_i \cdot W_1^i) W_2^i + o_i \quad (7)$$

For LLMs inference services, SLAs commonly employ two metrics [8]: i) Time to the First Token (TTFT), which refers to the latency incurred from a query is input until the first output token is generated, and ii) Time Per Output Token (TPOT), which measures the average time required to generate each subsequent output token after the first one. TTFT is related to *prefill stage* and TPOT is decided by *decode stage*.

B. Cloud-Device Inference System

Existing approaches to deploying model inference services in cloud-device systems can be broadly categorized into two categories: i) large-small model collaboration [9]–[12], which leverages the collaboration between large models deployed in the cloud and small models residing at the devices, and ii) computation offloading [13]–[17], which dynamically allocates computational tasks between the cloud and the devices.

For large-small model collaboration, one common technique is the early-exit mechanism, which allows the model to terminate its inference process prematurely when a certain level of confidence or accuracy threshold is met. For example, SPINN [9] dynamically selects the exit points and task division between devices and clouds based on resources and network conditions. Another common method is knowledge distillation, where a large, complex model in the cloud is

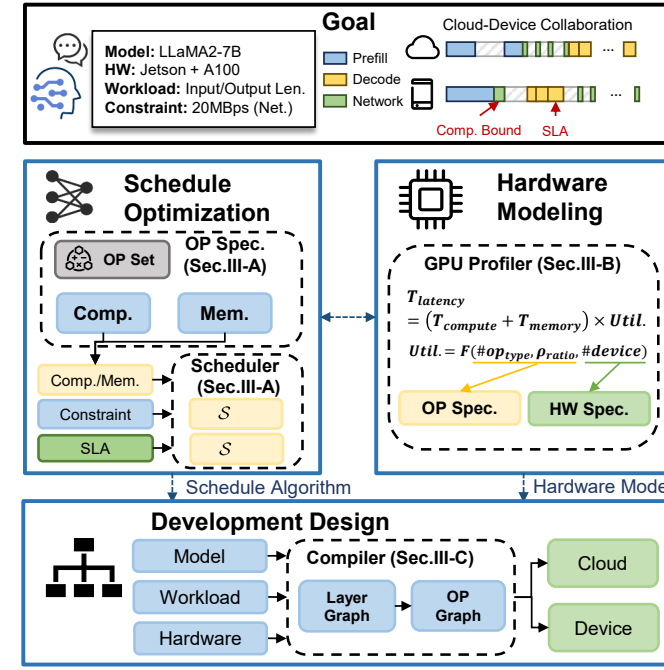


图 2. 我们设计的概览。我们的目标是确保推理延迟（TTFT 和 TPOT）在推理 SLA 范围内，同时最小化推理成本。

我们在图 2 中展示了该框架的概述，并总结了贡献如下：

- 1) 我们提出了一个用于在云设备系统中高效部署大型语言模型（LLM）的新框架。具体而言，我们在操作符粒度上分配计算负载，并提出了一种基于贪婪算法的成本-延迟平衡调度算法。
- 2) 我们开发了一个硬件仿真模型，可快速评估在不同硬件平台上具有不同计算与内存访问比的操作符的计算时间。
- 3) 我们提出了一个编译框架，便于对各种推理任务和云设备硬件组合进行定量评估和高效部署。实验结果表明，我们的方法满足推理延迟的需求，并相比仅在云端部署可将成本降低高达 20.7%。

二、背景与相关工作

A. 大型语言模型（LLMs）推理

在本节中，我们介绍了 LLM 的推理计算工作流程和 SLA。标准的 LLM 生成推理过程包括两个阶段 [6]: i) 预填充阶段，在该阶段中，使用给定的提示序列为 LLM 中的每个 Transformer 层生成键 / 值缓存（KV 缓存）；以及 ii) 解码阶段，该阶段利用并更新此 KV 缓存以逐个生成令牌，当前令牌的生成依赖于之前生成的令牌。在两个阶段中，每个令牌的生成都需要通过整个模型，通常由堆叠多个 Transformer 块构成 [7]。Transformer 块

由两个主要运算符组成：注意力运算符和前馈网络（FFN）运算符。

在预训练阶段，将第 i 层的输入特征表示为

$X_i \in R^{b \times s \times h}$ ，其中 b 表示批量大小， s 表示输入序列长度， h 表示 Transformer 的隐藏维度。对于注意力操作，计算为：

$$Q_i = X_i W^{Q_i}, K_i = X_i W^{K_i}, V_i = X_i W^{V_i} \quad (1)$$

$$A_i = \text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{h}}\right) V_i, O_i = A_i W^{O_i} + X_i \quad (2)$$

其中 $W^{Q_i}, W^{K_i}, W^{V_i}, W^{O_i} \in R^{h \times h}$ 是权重矩阵。FFN 运算符的计算方程为：

$$X_{i+1} = f_{act}(O_i W_1^i) W_2^i + O_i \quad (3)$$

其中 $W_1^i \in R^{h \times h'}$ 和 $W_2^i \in R^{h' \times h}$ 是权重矩阵（ h' 是第二个多层感知机层的隐藏维度）。

在解码阶段，每次迭代只针对当前的标记进行设计。与预训练阶段不同，解码过程中主要是矩阵 - 向量乘法（MV）操作，而不是矩阵 - 矩阵乘法（MM）。记第 i 层当前生成标记的特征为 $t_i \in R^{b \times 1 \times h}$ 。该过程的公式如下：

$$q_i = t_i \cdot W^{Q_i}, k_i = t_i \cdot W^{K_i}, v_i = t_i \cdot W^{V_i} \quad (4)$$

$$K_i \leftarrow \text{Concat}(K_i, k_i), V_i \leftarrow \text{Concat}(V_i, v_i) \quad (5)$$

$$a_i = \text{softmax}\left(\frac{q_i K_i^T}{\sqrt{h}}\right) V_i, o_i = a_i \cdot W^{O_i} + t_i \quad (6)$$

$$t_{i+1} = f_{act}(o_i \cdot W_1^i) W_2^i + o_i \quad (7)$$

对于大型语言模型（LLM）的推理服务，服务级别协议（SLA）通常采用两个指标 [8]: i) 首个令牌时间（TTFT），指从输入查询到生成第一个输出令牌所需的延迟；ii) 每个输出令牌时间（TPOT），用于衡量生成第一个输出令牌之后每个后续输出令牌所需的平均时间。TTFT 与预填充阶段相关，TPOT 由解码阶段决定。

B. 云端设备推理系统

在云 - 设备系统中部署模型推理服务的现有方法大致可以分为两类：i) 大小模型协作 [9]–[12]，—— 利用部署在云端的大模型与驻留在设备上的小模型之间的协作；ii) 计算负载分配 [13]–[17]，—— 在云端和设备之间动态分配计算任务。

对于大模型与小模型的协作，一种常见的技术是早期退出机制，该机制允许模型在达到一定的置信度或准确性阈值时提前终止推理过程。例如，SPINN [9] 根据资源和网络条件动态选择退出点以及设备和云之间的任务分配。另一种常见的方法是知识蒸馏，其中云端的大型复杂模型

TABLE I OPERATOR COMPUTATION, MEMORY ACCESS AND COMPUTATIONAL INTENSITY IN PREFILL AND DECODE STAGES.				
Operator Name	Stage	Computation (OPs)	Memory Access (Bytes)	Compute-to-Memory Access Ratio (OP/Byte)
$Q/K/V/O$ (Linear)	Prefill*	$s_{in}h^2$	$2s_{in}h + h^2$	$s_{in}h/(h + 2s_{in})$
	Decode**	h^2	$2h + h^2$	$h/(h + 2)$
$S \cdot V$ (Attention)	Prefill	s_{in}^2h	$2s_{in}h + s_{in}^2$	$s_{in}h/(2h + s_{in})$
	Decode	$(s_{in} + 1)h$	$h + (s_{in} + 1)h + s_{in} + 1$	$(s_{in} + 1)h/[h + (s_{in} + 1) \cdot (h + 1)]$
FFN	Prefill	$s_{in}hh'$	$s_{in}h + s_{in}h' + hh'$	$s_{in}hh'/[s_{in}(h + h') + hh']$
	Decode	hh'	$h + h' + hh'$	$hh'/(h + h' + hh')$
LM Head	Decode	hh_v	$2h + hh_v$	$hh_v/(2h + hh_v)$

* s_{in} represents the prefill size.

** The decode here is the first stage after the prefill stage.

used to train a smaller, more efficient model on the device-side. However, for LLMs, which are characterized by immense size and complexity, it is hard to construct a model supporting early-exit or distilling knowledge.

For computation offloading, the two primary methods are tensor parallelism and pipeline parallelism. For example, Co-Edge [13] proposes a tensor offloading strategy, which divides the computation workloads at the tensor level and allocates them between clouds and devices. Some works partition tasks in tensor and model levels [15]–[17], using tensor parallelism and pipeline parallelism simultaneously. However, these works cannot fully consider the different computation-to-memory access ratios of different operators and the dynamics of workloads, which compromises the efficiency of the service.

III. METHODOLOGY

In this section, we present our proposed framework for the efficient deployment of LLM inference services. We introduce our innovative scheduling strategy (Sec. III-A), tailored to dynamically allocate computational tasks between the cloud and devices, ensuring optimal performance under varying workloads. Then we elaborate on our hardware modeling and simulation methodology (Sec. III-B). Lastly, we outline our compilation and deployment system, which streamlines the process of adapting and deploying LLMs across diverse architectures (Sec. III-C).

A. Schedule Optimization Based on Operator Properties

Instead of layer-wise partition of the computational workload, we segment the load at operator granularity. That is because of the heterogeneity observed in the characteristics of various operators during the computation process of LLMs. When confronted with the SLAs violation imposed by resource-constrained devices, our algorithm would prioritizes the migration of operators with high computational intensity and memory access requirements to the cloud. This is driven by the fact that GPUs on the cloud side exhibit significantly higher utilization when handling operators with substantial workloads. This strategic relocation not only mitigates the resource limitations at the device-side, but also strives to minimize the associated cloud costs, ensuring that the SLA targets are met with minimal overhead.

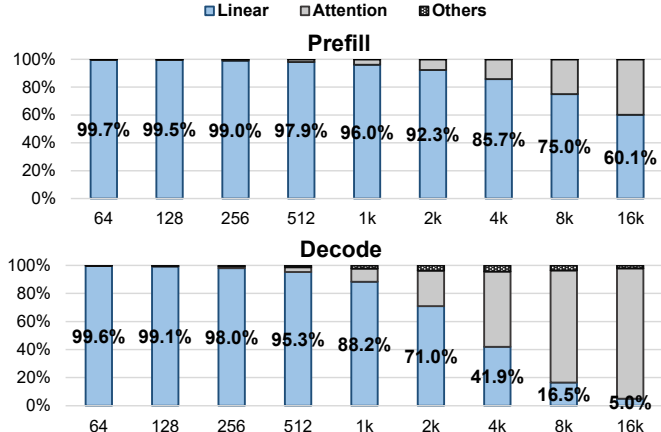


Fig. 3. The proportion of computing time requirements of different operators in the prefill and decode stages varies with the length of the processed text. The horizontal axis represents the length of the processed text.

Fig. 3 illustrates the computation timeshare of different operators in inference for LLMs. It can be observed that the linear and attention operators dominate the inference process and that the distribution is affected by the inference stages and the length of text. In the context of short text inference requests, linear operators predominantly contribute to the overall latency during both the prefill and decode phases. However, the share of attention computation gradually increases as the text sequence grows in length, with this trend being more pronounced in the decode stage.

Table I lists the equation of computation load, memory access volume and computation-to-memory access ratio for linear and attention operators. It can be observed that the computation load of attention operators is correlated with the square of the text length. In contrast, it grows linearly with the length of the text for linear operators, which also explains the change in the workload proportion of Fig. 3. Regarding the performance constraints, the prefill phase is characterized by a computational bottleneck, whereas the decode stage is marked by a bandwidth bottleneck. For prefill stage, the computation-to-memory access ratio is usually much greater than 1. In contrast, for decode stage, the computation-to-memory access ratio of the linear operators is approximately equal to 1 and independent of the text length. Similarly, as the text length increases, the computation-to-memory access ratio of the attention operators also converges to 1. Moreover,

表 I 预填充和解码阶段的操作符计算、内存访问和计算强度。

Operator Name	Stage	Computation (OPs)	Memory Access (Bytes)	Compute-to-Memory Access Ratio (OP/Byte)
$Q/K/V/O$ (Linear)	Pre ll*	$s_{in}h^2$	$2s_{in}h + h^2$	$s_{in}h/(h + 2s_{in})$
	Decode**	h^2	$2h + h^2$	$h/(h + 2)$
$S \cdot V$ (Attention)	Pre ll	s_{in}^2h	$2s_{in}h + s_{in}^2$	$s_{in}h/(2h + s_{in})$
	Decode	$(s_{in} + 1)h$	$h + (s_{in} + 1)h + s_{in} + 1$	$(s_{in} + 1)h/[h + (s_{in} + 1) \cdot (h + 1)]$
FFN	Pre ll	$s_{in}hh'$	$s_{in}h + s_{in}h' + hh'$	$s_{in}hh'/[s_{in}(h + h') + hh']$
	Decode	hh'	$h + h' + hh'$	$hh'/(h + h' + hh')$
LM Head	Decode	hh_v	$2h + hh_v$	$hh_v/(2h + hh_v)$

* s_{in} represents the pre ll size.

** The decode here is the rst stage after the pre ll stage.

用于在设备端训练一个更小、更高效的模型。然而，对于以巨大规模和复杂性为特征的大型语言模型（LLM）来说，很难构建一个支持早期退出或知识蒸馏的模型。

对于计算负载，有两种主要方法：张量并行和流水线并行。例如，Co-Edge 提出了一种张量负载策略，它在张量层面划分计算工作负载，并在云和设备之间分配它们。一些研究在张量和模型层面分割任务——同时使用张量并行和流水线并行。然而，这些研究无法充分考虑不同操作的计算与内存访问比例以及工作负载的动态变化，这会影

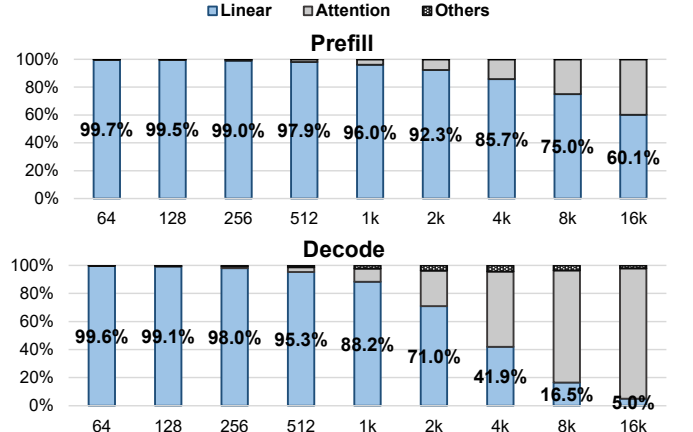


图 3. 不同算子在预处理和解码阶段的计算时间需求比例随处理文本长度的变化情况。横轴表示处理文本的长度。

图 3 展示了大语言模型推理中不同算子所占的计算时间份额。可以观察到，线性算子和注意力算子主导了推理过程，并且这一分布受推理阶段和文本长度的影响。在短文本推理请求的情况下，线性算子在预推理和解码阶段都对总体延迟贡献最大。然而，随着文本序列长度的增加，注意力计算的占比逐渐上升，这一趋势在解码阶段表现得更加明显。

表 I 列出了线性和注意力算子的计算负载、内存访问量以及计算与内存访问的比率的公式。可以观察到，注意力算子的计算负载与文本长度的平方相关。相比之下，对于线性算子，它随着文本长度线性增长，这也解释了图 3 中工作负载比例的变化。关于性能限制，pre ll 阶段以计算瓶颈为特征，而解码阶段以带宽瓶颈为特征。对于 pre ll 阶段，计算与内存访问的比率通常远大于 1。相比之下，对于解码阶段，线性算子的计算与内存访问比率约为 1，并且与文本长度无关。类似地，随着文本长度的增加，注意力算子的计算与内存访问比率也会收敛到 1。此外，

三、方法论

在本节中，我们提出了用于高效部署大语言模型推理服务的框架。我们介绍了我们的创新调度策略（第三节 A 部分），旨在动态分配云端与设备之间的计算任务，以确保在不同工作负载下的最佳性能。然后，我们详细说明了我们的硬件建模与仿真方法（第三节 B 部分）。最后，我们概述了我们的编译和部署系统，该系统简化了在多种架构上适配和部署大语言模型的过程（第三节 C 部分）。

A. 基于操作员属性的日程优化

我们不是按层对计算工作负载进行划分，而是按操作符粒度对负载进行分割。这是因为在大型语言模型的计算过程中，不同操作符的特性表现出异质性。面对资源受限设备施加的 SLA 违规情况时，我们的算法会优先将计算强度高且内存访问需求大的操作符迁移到云端。这是因为云端的 GPU 在处理大量工作负载时的利用率显著更高。这种战略性迁移不仅缓解了设备端的资源限制，还努力最小化相关的云成本，从而确保以最小的开销实现 SLA 目标。

Algorithm 1 Greedy-based algorithm for tasks scheduling on cloud-device system

```

1: Input: The LLM inference computation tasks set in current iteration  $\mathcal{S}$ , parameters of cloud and device platforms  $D_{cloud,device}$ , target latency  $\tau$ .
2: Output: Cloud-side tasks subset  $\mathcal{S}_{cloud}$ , device-side tasks subset  $\mathcal{S}_{device}$ .
3: Sort  $\mathcal{S}$  by linear and attention.
4:  $\mathcal{S}_{cloud} \leftarrow \emptyset$ 
5:  $\mathcal{S}_{device} \leftarrow \mathcal{S}$ 
6:  $T \leftarrow 0$ 
7: for  $s \in \mathcal{S}$  do
8:    $T = T + f(s, D_{device})$ 
9: end for
10: for  $s \in \mathcal{S}$  do ▷ Traverse linear first, then attention
11:   if  $T < \tau$  then
12:     break
13:   else
14:      $\mathcal{S}_{device} \leftarrow \mathcal{S}_{device} \setminus s$ 
15:      $\mathcal{S}_{cloud} \leftarrow \mathcal{S}_{cloud} \cup s$ 
16:      $T = T - f(s, D_{device}) + f(s, D_{cloud})$ 
17:   end if
18: end for
19: return  $\mathcal{S}_{cloud}, \mathcal{S}_{device}$ 

```

the computational complexity and memory access volume of operators during the prefill stage significantly exceed those in the decode stage, primarily due to the fact that the prefill stage processes multiple input tokens simultaneously, whereas the decode stage handles a single output token at a time.

Based on the above observations, we propose a greedy-based scheduling algorithm, as illustrated in Alg. 1. The objective of the algorithm is to allocate operators ($\mathcal{S}$) required by LLM within a specified iteration cycle, aiming to minimize the cost, i.e., minimize the use of cloud hardware, while satisfying latency constraints (τ). Initially, the algorithm assumes all tasks are assigned to the device ($\mathcal{S}_{device} = \mathcal{S}$) and estimates the total latency T using a function $f(s, D_{device})$. This function is obtained from hardware modelling, which computes latency considering operator and hardware parameters. Following this, the algorithm iterates through the task set, assessing whether the cumulative latency T would exceed the target τ under the current task assignment. If so, the algorithm offloads the current task to the cloud by moving it from $\mathcal{S}_{device}$ to $\mathcal{S}_{cloud}$ and updates the T . This process continues until the latency constraint τ is met. In this algorithm, we prioritize traversing and assigning linear operators to the cloud before attention operators. This decision stems from the strategic objective of migrating operators with higher computational and memory access requirements to the cloud.

Regarding the communication between the cloud and the device, when the cloud processes linear operators, only activations need to be transmitted, resulting in a relatively low communication volume that can be effectively concealed through pipelining. When the cloud computes attention op-

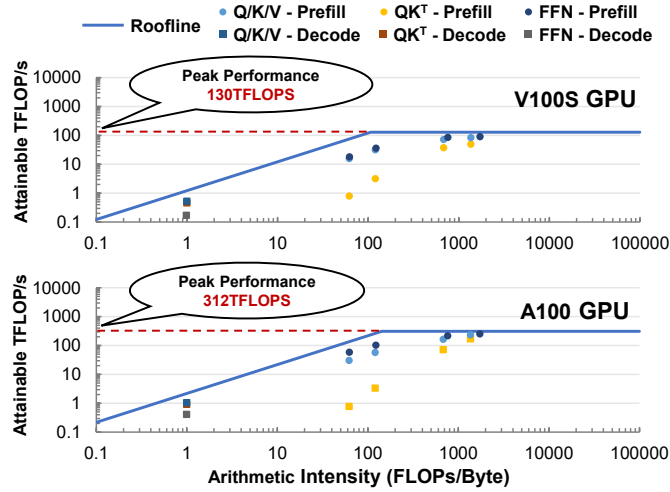


Fig. 4. NVIDIA V100S GPU and A100 GPU theoretical roofline curve and actual computational intensity of different operators.

erators, given that we have already prioritized the placement of linear layers on the cloud, the KV cache required by the attention operators is expected to be already resident in the cloud. As a result, only activations need to be transmitted maintaining a manageable communication overhead.

B. Hardware Modeling for The GPU

Modern GPU architectures are uniform in design and have commonalities in the operation mechanism. Considering the GPU’s pipeline design [18], the theoretical computation latency of the GPU can be calculated using the following Eq. 8:

$$\begin{aligned}
T_{GPU} &= \sum_{i=0}^{N_{op}} \max(T_{Compute}^{(i)}, T_{Memory}^{(i)}) \\
&= \sum_{i=0}^{N_{op}} \max\left(\frac{Operation_i}{P_i \times Parallelism_i}, \frac{Data Size_i}{BW_i}\right)
\end{aligned} \tag{8}$$

where N_{op} is the number of operators of the task, $Operation_i$ is the computation amount of different operations within the operator, P_i is the computing power of the operation specified by the GPU, and $Data Size_i$ indicates the amount of data accessed, and BW_i indicates the bandwidth of data accessed.

Fig. 4 shows the roofline model of the GPU and the attainable performance in running different inference operators. It is demonstrated that for operators with low computational intensity, the attainable performance of GPUs falls short of their theoretical peak performance. As a result, when theoretical models are used for derivations, substantial deviations can occur in inference tasks.

In this paper, we employ spline interpolation on empirically measured data to formulate an accurate hardware modeling approach. Specifically, by applying spline interpolation to the collected data points, we define a practical modeling formula that accurately reflects the hardware’s performance. Notably, under diverse task loads, the average absolute error of the simulation for a single operator is found to be 4.78%, demonstrating the effectiveness and accuracy of our modeling strategy.

算法 1 基于贪心的云设备系统任务调度算法

```

1: Input: The LLM inference computation tasks set in current iteration  $\mathcal{S}$ , parameters of cloud and device platforms  $D_{cloud,device}$ , target latency  $\tau$ .
2: Output: Cloud-side tasks subset  $\mathcal{S}_{cloud}$ , device-side tasks subset  $\mathcal{S}_{device}$ .
3: Sort  $\mathcal{S}$  by linear and attention.
4:  $\mathcal{S}_{cloud} \leftarrow \emptyset$ 
5:  $\mathcal{S}_{device} \leftarrow \mathcal{S}$ 
6:  $T \leftarrow 0$ 
7: for  $s \in \mathcal{S}$  do
8:    $T = T + f(s, D_{device})$ 
9: end for
10: for  $s \in \mathcal{S}$  do ▷ Traverse linear first, then attention
11:   if  $T < \tau$  then
12:     break
13:   else
14:      $\mathcal{S}_{device} \leftarrow \mathcal{S}_{device} \setminus s$ 
15:      $\mathcal{S}_{cloud} \leftarrow \mathcal{S}_{cloud} \cup s$ 
16:      $T = T - f(s, D_{device}) + f(s, D_{cloud})$ 
17:   end if
18: end for
19: return  $\mathcal{S}_{cloud}, \mathcal{S}_{device}$ 

```

在预填充阶段，操作符的计算复杂度和内存访问量明显超过解码阶段，这主要是由于预填充阶段同时处理多个输入令牌，而解码阶段一次处理一个输出令牌。

基于上述观察，我们提出了一种基于贪心的调度算法，如算法 1 所示。该算法的目标是在指定的迭代周期内分配 LLM 所需的操作符 ($\mathcal{S}$)，旨在最小化成本，即在满足延迟约束 (τ) 的前提下，尽量减少云硬件的使用。最初，算法假设所有任务都分配给设备 ($\mathcal{S}_{device} = \mathcal{S}$)，并使用函数 $f(s, D_{device})$ 来估算总延迟 T 。该函数通过硬件建模获得，考虑操作符和硬件参数计算延迟。随后，算法遍历任务集，评估在当前任务分配下累计延迟 T 是否会超过目标 τ 。如果是，算法会将当前任务卸载到云端，即将任务从 $\mathcal{S}_{device}$ 移至 $\mathcal{S}_{cloud}$ ，并更新 T 。此过程持续进行，直到延迟条件满足。

关于云与设备之间的通信，当云处理线性算子时，只需传输激活，这导致通信量相对较低，可以通过流水线有效隐藏。当云计算注意力操作时

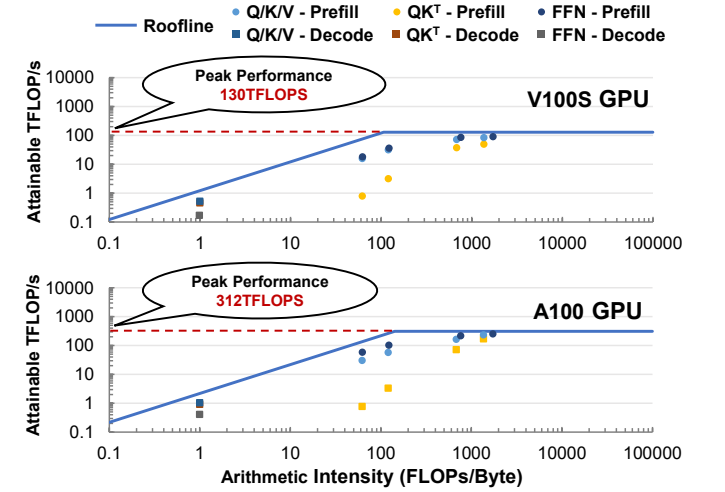


图 4. NVIDIA V100S GPU 和 A100 GPU 的理论峰值性能曲线以及不同算子的实际计算强度。

由于我们已经优先将线性层放置在云端，注意力算子所需的 KV 缓存预计已经驻留在云端。因此，只需要传输激活，从而保持可管理的通信开销。

B. GPU 的硬件建模

现代 GPU 架构在设计上是统一的，并且在操作机制上具有共性。考虑到 GPU 的流水线设计 [18]，GPU 的理论计算延迟可以使用以下公式 Eq.8 计算：

$$\begin{aligned}
T_{GPU} &= \sum_{i=0}^{N_{op}} \max(T_{Compute}^{(i)}, T_{Memory}^{(i)}) \\
&= \sum_{i=0}^{N_{op}} \max\left(\frac{Operation_i}{P_i \times Parallelism_i}, \frac{Data Size_i}{BW_i}\right)
\end{aligned} \tag{8}$$

其中 N_{op} 是任务的操作数， $Operation_i$ 是操作符内不同操作的计算量， P_i 是 GPU 指定操作的计算能力， $Data Size_i$ 表示访问的数据量， BW_i 表示访问数据的带宽。

图 4 显示了 GPU 的粗略模型以及在运行不同推理操作时可达到的性能。结果表明，对于计算强度低的操作，GPU 的可达性能低于其理论峰值性能。因此，当使用理论模型进行推导时，推理任务中可能会出现显著偏差。

在本文中，我们对实测数据采用样条插值方法来制定一种精确的硬件建模方法。具体来说，通过将样条插值应用于收集到的数据点，我们定义了一个能够准确反映硬件性能的实用建模公式。值得注意的是，在各种任务负载下，单个操作符的模拟平均绝对误差被发现为 4.78%，这展示了我们建模策略的有效性和准确性。

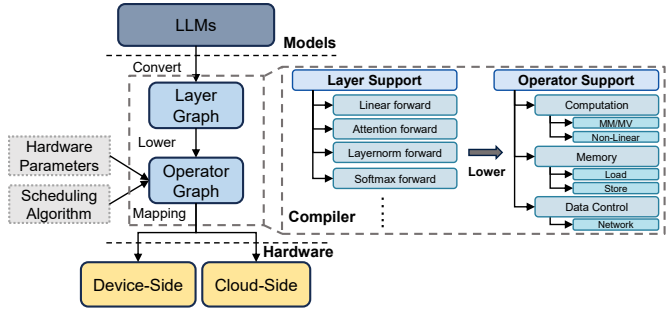


Fig. 5. Cloud-device LLMs compilation workflow.

C. Development Design for Cloud-Device Collaboration

In conjunction with the cloud-device collaboration system, LLMs' inference scenarios exhibit diverse and complex trends, primarily manifested in the interplay between hardware (device-side computing power and inference cost) and inference task (prefill size and decode size). This necessitates the deployment of convenient tool frameworks to facilitate software-to-hardware mapping.

We design a two-layer intermediate representative (IR) compilation process to achieve a more fine-grained optimization purpose. This process implements a structural analysis of the model, an operator dependency analysis, and an optimization of the cloud-device scheduling strategy, as summarized in Fig. 5.

In the layer computation graph IR, the compiler can realize the abstract representation of the computation layers and complete the dependency analysis. Further, the layer computation graph IR lowers for the operator computation graph IR, where the compiler will define specific operator computation properties, access properties, and data control properties for different computation monolayers, which can be directly utilized by the hardware model in Sec.III-B. The compiler also assigns the execution platform for each operator, adds network communication operations, and completes the inference scheduling optimization in conjunction with the algorithm mentioned in Sec. III-A.

After compilation, we get the set of tasks on the cloud-device computing platform. In addition, while obtaining the operator scheduling strategy, we can also calculate the inference performance of the cloud-device system based on the hardware model simulation.

IV. EXPERIMENT

A. Experiment Setup

Hardware Platform. To demonstrate the effect of the design works with different hardware combinations, we experimented with two different configurations:

a) Group-L: For the cloud-side platform, this configuration includes an NVIDIA V100S GPU with a peak memory bandwidth of 1134GB/s and a peak computing power of 130TFLOPS using Tensor Core at FP16 precision. The cost of use is 1.5×10^{-5} \$/s. For the device-side platform, we chose an NVIDIA Jetson Nano with a peak memory bandwidth of 25.6GB/s and a peak computing power of 0.47TFLOPS (GPU).

TABLE II
HARDWARE PARAMETERS OF CLOUD AND DEVICE PLATFORMS.

	Cloud-Side		Device-Side	
Platform	NVIDIA V100S	NVIDIA A100	NVIDIA Jetson Nano	Snapdragon 8s Gen 3
Frequency	1597MHz	1410MHz	1430MHz	921MHz
Computing Power	130TFLOPS (Tensor Core)	312TFLOPS (Tensor Core)	0.47TFLOPS (GPU)	8.50TFLOPS (GPU)
Bandwidth	1134GB/s	2039GB/s	25.6GB/s	76.6GB/s

b) Group-H: For the cloud-side platform, this configuration includes an NVIDIA A100 GPU with a peak memory bandwidth of 2039GB/s and a peak computing power of 312TFLOPS using Tensor Core at FP16 precision. The cost of use is 4×10^{-5} \$/s. We choose the Snapdragon 8s Gen 3 for the device-side platform with a peak memory bandwidth of 76.6GB/s and a peak computing power of 8.5TFLOPS (GPU).

Models. We use the the LLaMA2-7B [19]. The model's parameters for our experiments use precision in FP16 data format, with intermediate activations in FP16.

Key Metrics & Baseline. The simulator will evaluate the cloud-device collaboration system's latency and cost. We use device-side-only inference as a baseline for TTFT and TPOT and cloud-side-only inference as a baseline for inference cost. We observe the performance effects of hardware combination and inference workloads on cloud-device collaboration.

B. TTFT & TPOT Evaluation

Fig. 6 shows the inference latency by applying the method to deploy different operators of LLMs on the cloud-side and device-side. This experiment evaluates the TTFT and TPOT of different inference tasks. The results show that under the different hardware combinations and prefill/decode size, the TTFT and TPOT meet the needs of inference services (less than 400ms and 100ms).

C. Inference Cost Evaluation

Fig. 7 shows the cost reduction of the prefill and decode stages, achieved by applying our method to deploy different computing workloads for LLMs on the cloud-side and device-side. This experiment evaluated the cost of the prefill and decode stage for LLaMA2-7B. The results show that the computational costs of the prefill and decode stage for different inference tasks are reduced under different hardware combinations. The average end-to-end inference cost for both hardware combinations is calculated to be 11.6% and 20.7% lower than cloud-side inference, respectively.

V. CONCLUSIONS

In this paper, we propose a cloud-device collaborative inference framework for LLMs inference. We summarize the computation and memory access characteristics of operators and the characteristics of cloud-device hardware. Additionally, we verify the effect of the collaborative inference strategy under different cloud-device hardware combinations and different inference tasks using our compilation workflow. In the future, with the continuous improvement of computing power and bandwidth of devices, it is expected to achieve low latency and high throughput LLMs inference services with the help of the cloud-device collaborative inference framework.

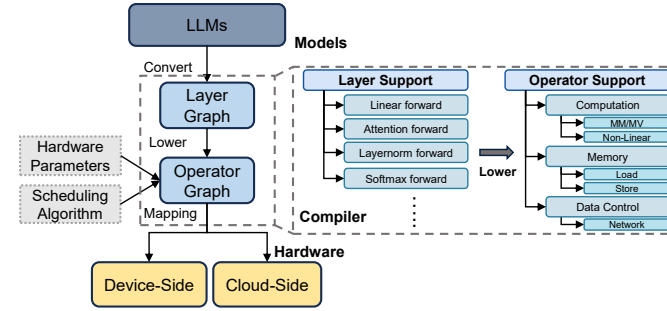


图 5. 云 - 设备大语言模型编译工作流程。

C. 云端与设备协作的开发设计

结合云 - 设备协作系统，LLM 的推理场景呈现出多样且复杂的趋势，主要表现为硬件（设备端计算能力和推理成本）与推理任务（预训练模型大小和解码大小）之间的相互作用。这就需要部署便捷的工具框架以促进软件到硬件的映射。

我们设计了一个两层的中间表示（IR）编译过程，以实现更细粒度的优化目标。该过程包括模型的结构分析、算子依赖分析以及云端 - 设备调度策略的优化，如图 5 所示。

在层计算图中间表示（IR）中，编译器可以实现计算层的抽象表示并完成依赖分析。此外，层计算图 IR 会向操作计算图 IR 降低，其中编译器将为不同的计算单层定义特定的操作计算属性、访问属性和数据控制属性，这些属性可以被第 III-B 节中的硬件模型直接利用。编译器还会为每个操作分配执行平台，添加网络通信操作，并结合第 III-A 节提到的算法完成推理调度优化。

编译后，我们得到云 - 设备计算平台上的任务集合。此外，在获取算子调度策略的同时，我们还可以基于硬件模型仿真计算云 - 设备系统的推理性能。

四、实验

A. 实验设置

硬件平台。为了展示该设计在不同硬件组合下的效果，我们实验了两种不同的配置：

a) Group-L: 对于云端平台，该配置包括一块 NVIDIA V100S GPU，峰值内存带宽为 1134GB/s，使用 FP16 精度的 Tensor Core 峰值计算能力为 130TFLOPS。使用成本为 1.5×10^{-5} \$/ 秒。对于设备端平台，我们选择了 NVIDIA Jetson Nano，其峰值内存带宽为 25.6GB/s，峰值计算能力为 0.47TFLOPS（GPU）。

表 II 云端和设备平台的硬件参数。

	Cloud-Side		Device-Side	
Platform	NVIDIA V100S	NVIDIA A100	NVIDIA Jetson Nano	Snapdragon 8s Gen 3
Frequency	1597MHz	1410MHz	1430MHz	921MHz
Computing Power	130TFLOPS (Tensor Core)	312TFLOPS (Tensor Core)	0.47TFLOPS (GPU)	8.50TFLOPS (GPU)
Bandwidth	1134GB/s	2039GB/s	25.6GB/s	76.6GB/s

b) 组 -H: 对于云端平台，该配置包括一块 NVIDIA A100 GPU，峰值内存带宽为 2039GB/s，使用 FP16 精度的张量核心峰值计算能力为 312TFLOPS。使用成本为 4×10^{-5} 美元 / 秒。我们为设备端平台选择了 Snapdragon 8s Gen 3，其峰值内存带宽为 76.6GB/s，峰值计算能力为 8.5TFLOPS（GPU）。

模型。我们使用了 LLaMA2-7B [19]。在我们的实验中，该模型参数使用 FP16 数据格式的精度，中间激活也使用 FP16。

关键指标与基线。模拟器将评估云 - 设备协作系统的延迟和成本。我们使用仅设备端推理作为 TTFT 和 TPOT 的基线，使用仅云端推理作为推理成本的基线。我们观察硬件组合和推理工作负载对云 - 设备协作性能的影响。

B. TTFT 和 TPOT 评估

图 6 显示了将该方法应用于在云端和设备端部署 LLM 的不同算子时的推理延迟。本实验评估了不同推理任务的 TTFT 和 TPOT。结果表明，在不同的硬件组合和预编码 / 解码大小下，TTFT 和 TPOT 均满足推理服务的需求（小于 400ms 和 100ms）。

C. 推理成本评估

图 7 显示了通过将我们的方法应用于在云端和设备端部署 LLM 的不同计算工作负载，实现的预推理和解码阶段的成本降低。本实验评估了 LLaMA2-7B 的预推理和解码阶段的成本。结果表明，在不同硬件组合下，不同推理任务的预推理和解码阶段的计算成本均有所降低。两种硬件组合的平均端到端推理成本分别比云端推理低 11.6% 和 20.7%。

五、结论

在本文中，我们提出了一种用于大型语言模型（LLMs）推理的云 - 设备协同推理框架。我们总结了算子计算和内存访问的特点以及云 - 设备硬件的特性。此外，我们使用我们的编译工作流程验证了在不同云 - 设备硬件组合和不同推理任务下协同推理策略的效果。未来，随着设备计算能力和带宽的持续提升，预计在云 - 设备协同推理框架的帮助下能够实现低延迟、高吞吐量的 LLMs 推理服务。

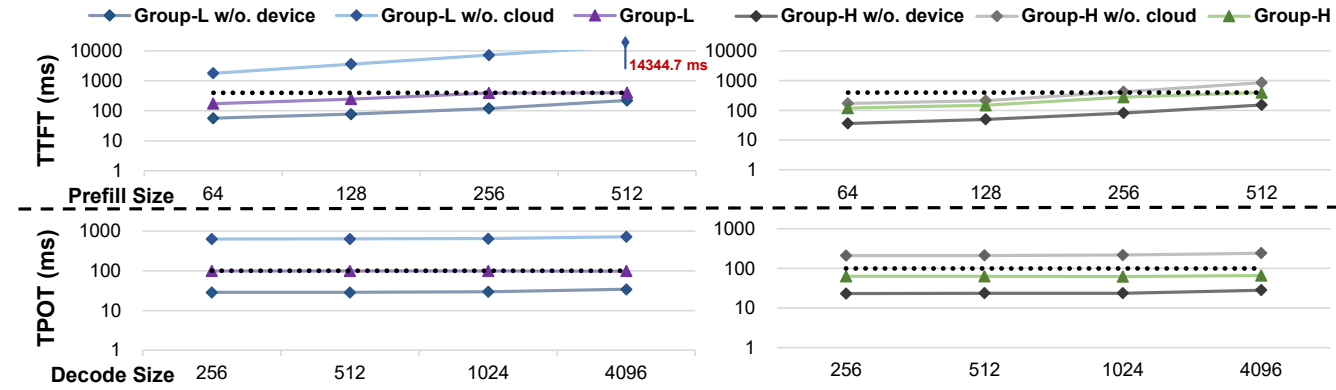


Fig. 6. TTFT and TPOT of Group-L and Group-H. The horizontal axis represents the prefill size or decode size. The vertical axis represents the latency.

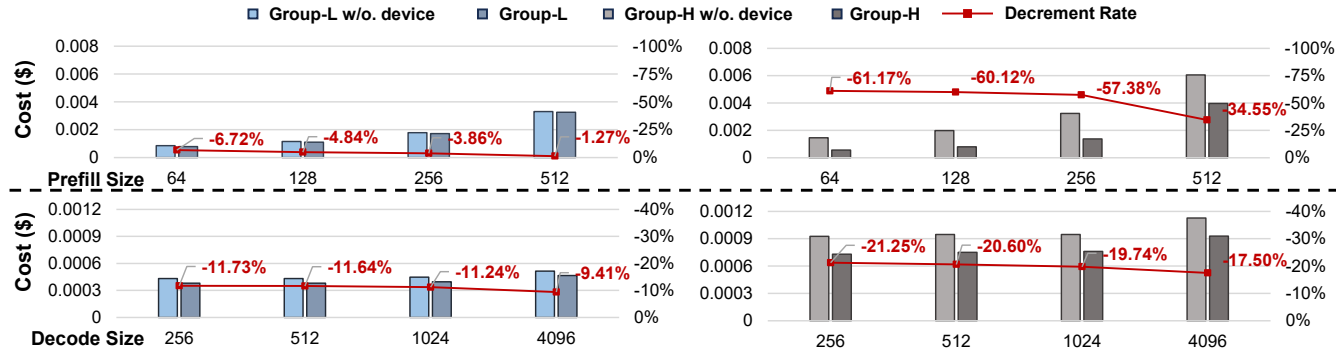


Fig. 7. Prefill stage and decode stage cost of Group-L and Group-H. The horizontal axis represents the prefill size or decode size. The right vertical axis represents the decrement rate of the cost.

ACKNOWLEDGMENT

This work was supported by the National Natural Science Foundation of China (No. 62325405, 62104128, U21B2031, 62204164), the National Key R&D Program of China (2023YFB4502200), Tsinghua EE Xilinx AI Research Fund, Tsinghua-Meituan Joint Institute for Digital Life, and Beijing National Research Center for Information Science and Technology (BNRist).

REFERENCES

- [1] C. Wang, J. Hu, C. Gao, Y. Jin, T. Xie, H. Huang, Z. Lei, and Y. Deng, “Practitioners’ expectations on code completion,” *arXiv preprint arXiv:2301.03846*, 2023.
- [2] S. Chen, M. Wu, K. Q. Zhu, K. Lan, Z. Zhang, and L. Cui, “Llm-empowered chatbots for psychiatrist and patient simulation: application and evaluation,” *arXiv preprint arXiv:2305.13614*, 2023.
- [3] C. Jeong, “A study on the implementation of generative ai services using an enterprise data-based llm application architecture,” *arXiv preprint arXiv:2309.01105*, 2023.
- [4] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [5] J. Yao, S. Zhang, Y. Yao, F. Wang, J. Ma, J. Zhang, Y. Chu, L. Ji, K. Jia, T. Shen *et al.*, “Edge-cloud polarization and collaboration: A comprehensive survey for ai,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 7, pp. 6866–6886, 2022.
- [6] Z. Zhou, X. Ning, K. Hong, T. Fu, J. Xu, S. Li, Y. Lou, L. Wang, Z. Yuan, X. Li *et al.*, “A survey on efficient inference for large language models,” *arXiv preprint arXiv:2404.14294*, 2024.
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [8] M. Agarwal, A. Qureshi, L. L. N. Sardana, J. Quevedo, and D. Khudia, “Llm inference performance engineering: Best practices,” 2023.

- [9] S. Laskaridis, S. I. Venieris, M. Almeida, I. Leontiadis, and N. D. Lane, “Spinn: synergistic progressive inference of neural networks over device and cloud,” in *Proceedings of the 26th annual international conference on mobile computing and networking*, 2020, pp. 1–15.
- [10] Y. Hu, Z. Li, Y. Chen, Y. Cheng, Z. Cao, and J. Liu, “Content-aware adaptive device-cloud collaborative inference for object detection,” *IEEE Internet of Things Journal*, 2023.
- [11] J. Yao, F. Wang, K. Jia, B. Han, J. Zhou, and H. Yang, “Device-cloud collaborative learning for recommendation,” in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 2021, pp. 3865–3874.
- [12] C. Ding, A. Zhou, Y. Liu, R. N. Chang, C.-H. Hsu, and S. Wang, “A cloud-edge collaboration framework for cognitive service,” *IEEE Transactions on Cloud Computing*, vol. 10, no. 3, pp. 1489–1499, 2020.
- [13] L. Zeng, X. Chen, Z. Zhou, L. Yang, and J. Zhang, “Coedge: Cooperative dnn inference with adaptive workload partitioning over heterogeneous edge devices,” *IEEE/ACM Transactions on Networking*, vol. 29, no. 2, pp. 595–608, 2020.
- [14] Y. Gong, Z. Jiang, Y. Feng, B. Hu, K. Zhao, Q. Liu, and W. Ou, “Edgerec: recommender system on edge in mobile taobao,” in *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, 2020, pp. 2477–2484.
- [15] S. Ye, J. Du, L. Zeng, W. Ou, X. Chu, Y. Lu, and X. Chen, “Galaxy: A resource-efficient collaborative edge ai system for in-situ transformer inference,” *arXiv preprint arXiv:2405.17245*, 2024.
- [16] Z. Yang, Y. Yang, C. Zhao, Q. Guo, W. He, and W. Ji, “Perllm: Personalized inference scheduling with edge-cloud collaboration for diverse llm services,” *arXiv preprint arXiv:2405.14636*, 2024.
- [17] M. Zhang, J. Cao, X. Shen, and Z. Cui, “Edgeshard: Efficient llm inference via collaborative edge computing,” *arXiv preprint arXiv:2405.14371*, 2024.
- [18] J. G. Cornelis and J. Lemeire, “The pipeline performance model: a generic executable performance model for gpus,” in *2019 27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*. IEEE, 2019, pp. 260–265.
- [19] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.

致谢

本工作得到了中国自然科学基金（编号：62325405、62104128、U21B2031、62204164）、中国国家重点研发计划（2023YFB4502200）、清华大学电子工程系赛灵思人工智能研究基金、清华-美团数字生活联合研究院以及北京信息科学与技术国家研究中心（BNRist）的支持。

参考文献

- [1] C. Wang、J. Hu、C. Gao、Y. Jin、T. Xie、H. Huang、Z. Lei 和 Y. Deng, 《从业者对代码补全的期望》, arXiv 预印本 arXiv:2301.03846, 2023.
- [2] S. Chen、M. Wu、K. Q. Zhu、K. Lan、Z. Zhang 和 L. Cui, 《用于精神科医生与患者模拟的 LLM 驱动聊天机器人: 应用与评估》, arXiv 预印本 arXiv:2305.13614, 2023.
- [3] C. Jeong, 《基于企业数据 LLM 应用架构的生成式 AI 服务实现研究》, arXiv 预印本 arXiv:2309.01105, 2023.
- [4] T. Brown、B. Mann、N. Ryder、M. Subbiah、J. D. Kaplan、P. Dhariwal、A. Neelakantan、P. Shyam、G. Sastry、A. Askell 等, 《语言模型是少样本学习者》, 神经信息处理系统进展, 第 33 卷, 第 1877–1901 页, 2020.
- [5] J. Yao、S. Zhang、Y. Yao、F. Wang、J. Ma、J. Zhang、Y. Chu、L. Ji、K. Jia、T. Shen 等, 《边缘-云的极化与协作: 一个共 ...

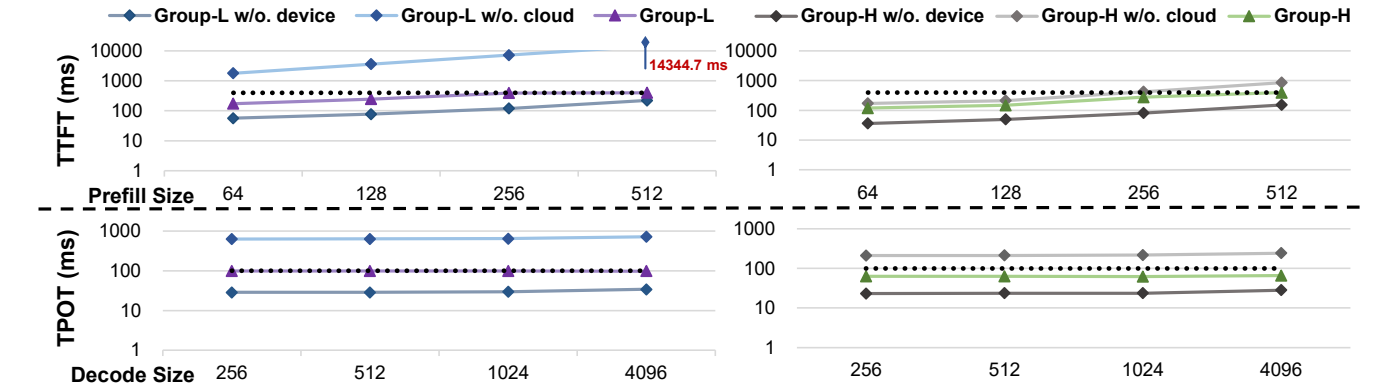


Fig. 6. TTFT and TPOT of Group-L and Group-H. The horizontal axis represents the pre ll size or decode size. The vertical axis represents the latency.

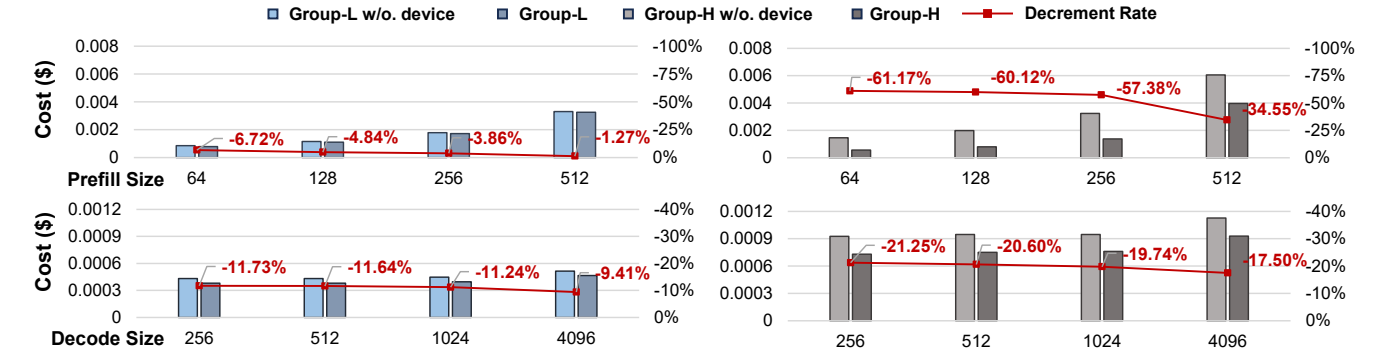


图 7. Group-L 和 Group-H 的预解码阶段和解码阶段成本。横轴表示预解码大小或解码大小。右侧纵轴表示成本的下降率。

[9] S. Laskaridis, S. I. Venieris, M. Almeida, I. Leontiadis, 和 N. D. Lane, “Spinn: 在设备和云上的神经网络协同渐进推理,” 发表于第 26 届年度国际移动计算与网络会议论文集, 2020, 第 1–15 页。

[10] Y. Hu, Z. Li, Y. Chen, Y. Cheng, Z. Cao, 和 J. Liu, “面向内容感知的自适应设备-云协作推理用于对象检测,” IEEE 物联网学报, 2023. [11] J. Yao, F. Wang, K. Jia, B. Han, J. Zhou, 和 H. Yang, “用于推荐的设备-云协作学习,” 发表于第 27 届 ACM 知识发现与数据挖掘会议论文集, 2021, 第 3865–3874 页。[12] C. Ding, A. Zhou, Y. Liu, R. N. Chang, C.-H. Hsu, 和 S. Wang, “认知服务的云边协作框架,” IEEE 云计算学报, 第 10 卷, 第 3 期, 第 1489–1499 页, 2020. [13] L. Zeng, X. Chen, Z. Zhou, L. Yang, 和 J. Zhang, “